C# Chapter 3 – Practical Assignment 2

Inheritance • Polymorphism • Interfaces • Abstraction

Instructions

- Write a separate C# program for each question.
- Use meaningful class, property, and method names.
- Create the required objects and display the results.
- Use constructors wherever specified.

Section 1: Inheritance

1. Employee Management System

Create an employee management program using inheritance.

- Create a base class **Employee** with Name, EmployeeId, and Salary properties, a constructor, and DisplayEmployeeDetails().
- Create **Developer** derived from Employee with a ProgrammingLanguage property and DisplayDeveloperDetails().
- Create **Manager** derived from Employee with a TeamSize property and DisplayManagerDetails().
- Use base() to initialize the inherited properties. Create objects of Developer and Manager and display their details.

2. Vehicle Management System

Create a vehicle program using a base class and derived classes.

- Create a base class **Vehicle** with Brand, Model, and Year properties, a constructor, Start(), and Stop().
- Create **Car** with NumberOfDoors and DisplayCarDetails().
- Create **Bike** with EngineCapacity and DisplayBikeDetails().
- Use base() in both derived-class constructors. Call the inherited Start() and Stop() methods using the derived objects.

Section 2: Constructor Inheritance and base()

3. Person and Student

Create a student information program using constructor inheritance.

- Create a base class **Person** with Name, Age, and Address properties and a constructor.
- Create a derived class **Student** with RegisterNumber, Course, and Marks properties.
- Use base() to initialize the Person properties from the Student constructor.
- Create at least two Student objects and display all inherited and student-specific details.

4. Product and ElectronicProduct

Create an electronic product program using inheritance.

- Create a base class **Product** with ProductId, ProductName, and Price properties and a constructor.
- Create **ElectronicProduct** derived from Product with Brand and WarrantyInYears properties.
- Use base() to initialize the base-class properties.
- Implement CalculateDiscount(double percentage), which returns the price after discount.
- Implement DisplayDetails(). Create at least two ElectronicProduct objects and display their details and discounted prices.

Section 3: Method Overriding and Runtime Polymorphism

5. Shape – Calculate Area

Create a shape program using method overriding.

- Create a base class **Shape** with a virtual method CalculateArea().
- Create derived classes **Rectangle** with Length and Breadth, and **Circle** with Radius.
- Override CalculateArea() in both derived classes to calculate and display their respective areas.
- Create objects of Rectangle and Circle and call CalculateArea().

6. Payment System – Runtime Polymorphism

Create a payment program that demonstrates runtime polymorphism.

- Create a base class **Payment** with an Amount property, a constructor, and a virtual ProcessPayment() method.
- Create **CreditCardPayment**, **UPIPayment**, and **CashPayment** derived classes.
- Override ProcessPayment() in each derived class with a payment-method-specific message.
- Create derived-class objects and store them in Payment references. Call ProcessPayment() through the base-class references.

Section 4: Interfaces and Multiple Inheritance

7. Notification System

Create a notification system using an interface.

- Declare an interface **INotification** with SendNotification(string message).
- Create EmailNotification, SMSNotification, and PushNotification classes that implement INotification.
- Implement SendNotification() differently in each class.
- Create objects of all three classes and send different messages.

8. Payment Gateway – Multiple Inheritance Using Interfaces

C# does not support inheriting from multiple classes. It supports a form of multiple inheritance through implementing multiple interfaces. Demonstrate this using the following requirements:

- Declare an interface **IPayment** with Pay(double amount).
- Declare a second interface **IRefund** with Refund(double amount).
- Create **RazorpayPayment** that implements both IPayment and IRefund.
- Create **StripePayment** that also implements both IPayment and IRefund.
- Implement both methods in each class with provider-specific messages.
- Create objects of both classes and call Pay() and Refund().

Section 5: Abstraction

9. Smart Meter Management System

Create a smart meter program combining inheritance, overriding, and interfaces.

- Create a base class **Meter** with SerialNumber and Reading properties, a constructor, DisplayMeterDetails(), UpdateReading(), and a virtual CalculateBill().
- Create **ElectricMeter** with Voltage and **GasMeter** with Pressure.
- Override CalculateBill() in both derived classes using different sample rates.

- Declare an interface **IMeterOperations** with ActivateMeter() and DeactivateMeter().
- Implement IMeterOperations in both derived classes. Create objects, display details, update readings, calculate bills, and activate/deactivate the meters.

10. Abstract Class – Employee Salary Management

Create an employee salary program using abstraction.

- Create an abstract class **Employee** with Name and EmployeeId properties and a constructor.
- Declare an abstract method **CalculateSalary()** that returns a double.
- Create a derived class **FullTimeEmployee** with a MonthlySalary property.
- Create another derived class **PartTimeEmployee** with HoursWorked and HourlyRate properties.
- Override CalculateSalary() in FullTimeEmployee to return MonthlySalary.
- Override CalculateSalary() in PartTimeEmployee to return HoursWorked multiplied by HourlyRate.
- Create objects of both derived classes and display their names, employee IDs, and calculated salaries.

Remember: *An abstract class cannot be instantiated directly. An abstract method has no implementation in the abstract class and must be overridden by a non-abstract derived class.*